CENG322 **Spring 2025**

PROGRAMMING ASSIGNMENT 3

Due date: 28.05.2024 23:00

In this programming assignment, you will implement a multi-threaded task scheduler that simulates multiple producer and consumer threads operating on bounded queue buffers with priorities.

Your program should have a task scheduling system with the following structures:

Producer threads: Generate tasks (like items we have in our lecture) randomly with different priority levels.

Consumer threads: Consume the highest-priority available tasks from the queues.

Task queues with three priority levels: One bounded circular queue to hold tasks for each priority (0 = High, 1 = Medium, 2 = Low). Each task may have an identifier and producer info.

Synchronization structures for each task queue: A mutex for mutual exclusion to the queue, a semaphore to hold the number of tasks in the queue, a semaphore to hold available slots in the queue.

You need to implement a task scheduling simulation with the following properties:

- Producers create tasks with random priorities, wait if the queue (of that specific queue) is full, and enqueue the task if there is space.

- Consumers try to fetch tasks from the highest-priority non-empty queue. They consume whenever there is a task in any of the queues.

- Use *sleep()* function to simulate time delays in task generation and processing.

You should use POSIX threads, mutex locks, and semaphores (no other mechanisms like condition variables etc.) to implement parallelism and synchronization in your program.

Your program should print logs of all enqueue and dequeue operations with thread ID, task ID, and priority.

Your program should accept the number of producers, number of consumers, max queue size (fixed for each priority level), number of tasks as a command-line argument. Accordingly, it must generate the data structures.

Your code should have mainly three functions with given flows:

1) producer_thread:

```
while (number of tasks to be simulated){
     produce a task and randomly assign a priority
     whenever there is an empty slot
          enqueue the task in the appropriate task queue
          sleep for some time to simulate the work (like 100ms)
}
```

2) consumer_thread:

```
while (true){
     for all queues starting from the highest-priority
          whenever there is a task
          dequeue task
          sleep for some time to simulate the work (like 150ms)
}
// you should have a mechanism to inform all tasks (number of
tasks) have been consumed
```

3) main:

```
Allocate memory for data structures given as arguments
Create producer threads
Create consumer threads

Wait for all producers finished
Exit when all tasks are consumed
```

Notes:

- You need to work individually, no group work is allowed.
- Submit your extensively commented source code via Teams. Please create a compressed file including all source files; and name it as yourstudentnumber_P3.zip (e.g. If your student number is 201812345678, the file name must be 201812345678_P3.zip).
- No late homework will be accepted.
- You are required to have a demonstration in the following weeks. Your TA will run and test your program, and ask questions about your implementation.